Longitude — plan your trip from orbit

Generated 29 September 2026 · Longitude · Next.js 16 · React 19 · CesiumJS · PostgreSQL

Longitude is a tour and travel website built around a live 3D Earth. You open the site on a globe in space, it works out where you are, you pick where you want to go, and scrolling flies the camera along the real flight path, down into a photorealistic 3D city and between its landmarks. From there you browse curated tours, open one, choose seats and book it. Accounts are protected with a password and a 6-digit code sent to your email.

Stack at a glance: Next.js 16 (App Router, Turbopack) · React 19 · TypeScript · Tailwind CSS v4 · CesiumJS 1.145 with Cesium ion (Google Photorealistic 3D Tiles, terrain, imagery) · GSAP 3.15 (ScrollTrigger, Flip, SplitText) · hand-written WebGL2 shaders · PostgreSQL · Prisma 7 · Zod · Nodemailer.

For the full inventory of every library, service, API route, file and technique, read `docs/PROJECT_GUIDE.md`.

Contents

1. Quick start
2. Environment variables
3. How the site works, page by page
4. Accounts and the email code (OTP)
5. How a request flows through the app
6. Scripts
7. Project structure
8. Degraded modes
9. Troubleshooting
10. Content and credits

Quick start

You need **Node.js 20 or newer** (the project is developed on Node 24). Docker is optional.

```
npm install          # installs packages, copies CesiumJS into public/cesium, generates the Prisma client
```

Start the database in its own terminal and leave it running:

```
npm run db:local     # a real PostgreSQL on port 5433, no Docker needed (data in ./.data/postgres)
```

The first time only, create the tables and load the sample catalog (in a second terminal):

```
npm run db:setup     # applies the migrations, seeds 8 destinations, 38 attractions and 16 tours
```

Then run the site:

```
npm run dev          # http://localhost:3000
```

The 3D Earth renders straight away, even with no keys and no database. Without a Cesium ion token the globe uses NASA imagery on a smooth sphere and a notice explains how to turn on the 3D city. Without a database the destinations and tours still show (from the bundled catalog); only bookings and accounts need Postgres.

Turn on the 3D city (free)

1. Create a free account at ion.cesium.com (Community plan).
2. Under **Access Tokens**, copy your default token.
3. Put it in `.env.local` as `NEXT_PUBLIC_CESIUM_ION_TOKEN`, then restart `npm run dev`.

Turn on real emails for the sign-in code

Signing up and signing in both email a 6-digit code. In development you don't need a mail account: when `SMTP_HOST` is blank the code is **printed in the terminal running** `npm run dev` and also shown under the code boxes on the page (with a button that types it in for you). To send real emails, fill in the SMTP settings. With Gmail:

1. Turn on 2-Step Verification for your Google account.
2. Create an App Password at myaccount.google.com/apppasswords.
3. In `.env.local` set `SMTP_HOST=smtp.gmail.com`, `SMTP_PORT=465`, `SMTP_USER=you@gmail.com` and `SMTP_PASS=` the 16-character app password. Restart `npm run dev`.

Any SMTP provider works the same way (Resend, Brevo, Mailgun, SendGrid, Mailtrap for testing). In production the site refuses to issue codes until SMTP is configured, so nobody is ever shown a code on screen.

Environment variables

Copy `.env.example` to `.env.local` and fill in what you need. `.env.local` is never committed.

Variable	Needed?	What it does
`NEXT_PUBLIC_CESIUM_ION_TOKEN`	For the 3D city	Cesium ion token: Google Photorealistic 3D Tiles, World Terrain, World Imagery, OSM Buildings (fallback) and Google geocoding for search. Free; used in the browser, so it is public by design (restrict it to your site's URLs in ion).
`GOOGLE_MAPS_SERVER_API_KEY`	No, leave blank	Optional own Google Geocoding + Places key. Blank means search uses Google geocoding through Cesium ion, then Open-Meteo / OpenStreetMap Nominatim; attractions for other places come from Wikipedia.
`DATABASE_URL`	For bookings and accounts	PostgreSQL connection string. The default matches `npm run db:local` and `npm run db:up` (port 5433).
`AUTH_SECRET`	In production	Signs the session cookie and hashes the email codes. A development-only fallback is used when blank. Generate one with `node -e "console.log(require('crypto').randomBytes(32).toString('base64url'))"`.

Variable	Needed?	What it does
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	For real emails	The mail server that sends the 6-digit codes. Blank in development prints codes in the terminal.
`SMTP_SECURE`	No	`true` forces implicit TLS. Defaults to on for port 465, STARTTLS otherwise.
`MAIL_FROM`	No	The sender, e.g. `Longitude <codes@yourdomain.com>`. Defaults to `SMTP_USER`.

How the site works, page by page

Every page shares the same frame: a fixed navigation bar with a live UTC clock, a footer, a subtle film grain, and two page transitions that play when you follow certain links (see Transitions).

Home `/` — the 3D journey

The home page is one long scroll-driven story over a live CesiumJS globe that stays pinned to the screen.

1. **Earth.** The planet from orbit, lit so there is always a day side and a night side with city lights.
2. **Your location.** Your browser's location (only if you allow it) or a city you type becomes the origin. If you say no, the site guesses a city from your time zone and labels it as approximate.
3. **Where do you want to go?** Pick one of the 8 curated destinations or search for any place.
4. **Route.** The camera flies the great-circle path (the curved route aircraft really fly) while a small plane travels along an arc drawn above the surface.
5. **Arrival and attractions.** The camera drops into the photorealistic 3D city and glides between landmarks, one per scroll step, with a panel describing each.
6. **Tours.** The destination's bookable tours.

Scrolling is mapped to camera positions by GSAP ScrollTrigger; a `CameraDirector` smooths the camera toward each target every frame, so scrubbing back and forth stays fluid. Below the journey come the destination dossier and marquee, the destination's tours, a reel of all destinations and a "How it works" section.

Destinations `/destinations` — the flight atlas

A dot-matrix world map stays pinned while the destination list scrolls past; each destination draws its flight route from Chandigarh (IXC) on the map. Hovering a name briefly previews its photo. Arriving here from another page plays the **jet transition**: a realistic 3D airliner flies across the screen and its wake opens onto the new page.

A destination `/destinations/[slug]` — the orbital survey

The city as a survey seen from orbit: one continuous WebGL world (three.js) built from the **real land** under the city, a square of terrain 9 to 39 km across sampled from open elevation data, drawn as a point cloud, contour lines and a fine grid in white on black, with amber reserved for the instruments. Scrolling flies one camera through it, chapter by chapter:

1. **Acquire.** The land scans in from the horizon under a low sun, and the city's name stands over it in large mono type with its survey number, coordinates, number of places and route length.
2. **Brief.** The camera rises and slowly circles the contoured land; the city's description, best season, time zone and elevation sit beside its photo, which decodes from coarse blocks into the picture.

3. **Strata.** The land is sliced by stacked translucent planes, one per landmark, that drift apart. Their labels hang on leader lines at the right; hovering one lights where its plane cuts the land, clicking flies there.
4. **Each landmark.** The camera flies to the landmark's pin. Its photo stands above it on a panel that decodes from large blocks to sharp, in monochrome until the cursor is on it. A callout gives its category, name, coordinates and elevation; best time and how long to plan sit at the right.
5. **Route.** Seen from straight above, an amber route draws itself through the landmarks in visiting order, with its elevation profile drawn underneath.

Then the survey draws back into a slowly turning object behind the city's tours, which rise into place: titles reveal word by word, each row's rule draws in, thumbnails decode, prices count up, and the duration fills a small slider (photos turn to colour on hover; a click plays the boarding transition).

Instruments stay at the edges: the region, coordinates and local time at the top left; a chapter panel with progress fills (click a chapter to fly there), the altitude, heading, elevation range and grid of the camera; "[Fly there in 3D]" and the terrain credit. Wherever the cursor rests on the land a probe reads out the latitude, longitude and real elevation under it. With reduced motion the chapters cut in place instead of flying.

It never goes blank. If the browser takes the GPU away (a driver reset, too many WebGL contexts in the tab) the page keeps drawing the same chapters with a Canvas 2D renderer, rebuilds WebGL when the browser gives the context back, or tries again on a fresh canvas. Without WebGL at all, the 2D renderer carries the whole page. If a survey file is missing, a generated landscape stands in. A frame-rate governor lowers the render resolution on slow machines.

The terrain is prepared ahead of time by `npm run survey:build` (see Scripts), which writes `public/survey/<slug>.json` for each destination. Elevation data: Mapzen / AWS Terrain Tiles (SRTM, GMTED, ETOPO1, EU-DEM and others).

Tours `/tours`

1. **The wall.** A tilted 3D wall of every tour's photos drifting in columns, under the headline.
2. **The route.** On wide screens the section pins and scrolls sideways along a flight path of cities; each tour is a boarding pass, and the page's colours morph to each city's theme as you pass it.
3. **The timetable.** All 16 tours side by side as a clean index: photo, route, length as day pips, pace as a three-bar meter, group size and price. Sort by route, days or price (rows glide to their new places) or press **Surprise me** to have a light run down the list and pick a flight for you.
4. **Arrivals.** What every tour includes.

Clicking a tour plays the **boarding transition**: the tour's photo lifts off the page and opens to fill the screen, then hands over to the tour page.

A tour `/tours/[slug]`

Each tour page wears its city's colours and reads like a journey: the hero photo with a boarding pass, a brief that lights up word by word, **the prints** (a 3D jet crosses the section, its contrail tears the sky open onto a photo, the photo becomes a print and the rest fan out and take turns in the light), the day-by-day plan with an odometer, the hotel as a room key card that flips, what's included, and **Final call**: a boarding pass where you pick seats on a cabin map and the fare, GST (5%) and total roll to the new numbers. A slim booking bar follows you down the page.

Booking `/book/[slug]` → `/book/confirmation/[reference]`

The booking page is a departures terminal at night, in the tour city's colours on warm black. A departures board lists this flight and the next five in clear mono type, its rows rolling in as the page opens (each one links to its own booking page), and a check-in form in four steps (lead traveller, departure, travellers and seats, requests) fills in a boarding pass beside it as you go: the name as you type it, dates and seats that roll in when they change, a total that counts up to the new figure; each typed character rises out of a soft blur. The form keeps a draft per tour for the session, checks each field as you leave it, and lists anything missing with links straight to it. The calendar works from the keyboard and lights up the whole trip.

It posts to `/api/bookings`. The server re-checks everything, prices it (price × travellers + 5% GST) and saves it with a reference like `LGT-7K3Q-9XA` (an alphabet without I, L, O or U, so it reads clearly over the phone). Then the screen shuts into a board of blank flaps, in a wave out from the button; the board says `LG 109 · CHECKED IN` and the reference, and falls open onto the confirmation, where the reference rolls in a character at a time and the pass prints out of a slot. There's an `.ics` calendar file and a print-ready pass. If you were signed in, the booking is attached to your account.

Account `/account`

Signed out, the page is a sign-in screen over a live WebGL Earth horizon whose sun rises and sets. The form is a glass boarding pass with two tabs, **Sign in** and **Create account**, and a second step for the emailed code (below). Signed in, it shows your name, your trips and a sign-out button. Anyone can use **Find a booking** (reference + email) for bookings made without an account.

About `/about`

One full-screen stage and one story scrolled through it. A grey moon comes alive as the Earth: life spreads out from Chandigarh, where Longitude began, behind a glowing, torn edge, while the planet turns and tilts to 23.4°. The camera then falls towards the Himalaya, levels out over the dusk line with the air glowing on the horizon, and dives through cloud. It comes out at dusk above a sea of cloud, where the story is told. Then a satellite, Landsat 8, glides in out of the sun among three lenses of glass that explain how the site works, and shines its beam down on the cloud, drawn the way each lens sees the world: an imaging swath with a line sweeping down it (the globe), tiles lighting up as they stream in (3D cities), radar rings spreading out (terrain). Scrolling steps through them; hovering a lens takes over, a click keeps it, and the glass magnifies whatever is under the pointer. Night falls while it's there: the sun sinks into the cloud, the satellite keeps the last sunlight until the Earth's shadow climbs over it, and the stars and a crescent moon come out over a moonlit sea of cloud. It turns slowly by itself; grab it to spin it, tap it to make its beam flare. Then the camera sinks through the cloud into the Kullu valley, a living photo that shifts in depth with the pointer while mist drifts through it; it draws into a porthole that flies back to orbit and lands on the Earth as a point of light. The planet is then yours: drag it to turn it, hover to read where you are, tap to drop a pin and plan a trip there. Its stars twinkle, a third of them visibly. Meanwhile its day goes round: night sweeps across the face and the cities light up, the sun slips behind the planet's limb and comes out again in a flare, and the Moon circles it, crescent to full, each throwing its shadow on the other when they line up. **Earth.** and the ways to reach a planner sit beside it; the cancellation policy follows below.

Everything is drawn live in one WebGL2 canvas (three.js), and the words over it follow the same damped scroll position, so they always move together. The cloud is ray-marched; fog rises through the frame to hide each cut and lifts away again; the lenses' glass is a copy of the scene drawn under each card. With reduced motion the chapters cut between still frames; without WebGL the words play over a still of the Earth. The old links

(`/about#how-it-works`, `#contact`, `#policies`) land on their chapters. The satellite is NASA's public-domain Landsat 8 model, packed for the web into one 0.8 MB file (`public/about/landsat8.bin`).

Transitions

- **Jet transition** (links to `/destinations`): uses the browser's View Transitions API plus a WebGL2 airliner rendered in real time (physically based lighting, clear coat, navigation lights). The new page is revealed through the widening wake behind the wings.
- **Boarding transition** (links to a tour): the clicked photo expands to full screen, the route and tour name rise in, and the tour page takes over once its hero photo is ready.

Both are skipped for people who ask their system for reduced motion, and both have watchdogs so a slow page can never leave you stuck mid-transition.

Accounts and the email code (OTP)

Every account is tied to an email address the owner has proven they can read.

Creating an account

1. You enter a name, email and password. The server checks them, makes sure the email isn't already used, hashes the password and emails a 6-digit code. **No account exists yet.**
2. You type the code into six boxes on the pass. If it's right, the account is created (marked as email-verified) and you're signed in.

Signing in

1. You enter your email and password. If they're right, a code is emailed.
2. You type the code. If it's right, the session starts.

What you see

- Typing: each digit pops into its box; the next box shows a blinking caret. Paste works, and a full code is checked straight away while a light sweeps across the boxes.
- Wrong code: the row shakes its head (a wiggle that dies away) and tints red, then every box turns over like a departures-board tile, left to right: the top half of the digit falls forward on its hinge and a red **×** swings down into place. A beat later the crosses turn over again to blank tiles, the red fades and the caret comes back for another try. The message decodes in, and one of the five "tries" pips flashes red and drains. The little plane in the pass header turns red.
- Expired or out of tries: the tiles turn over to dashes and stay there, and the main button becomes **Send a new code**.
- Right code: the boxes fill amber left to right, the digits lift away, the boxes slide together into one round seal and a tick draws itself inside it. Then the pass is stamped "cleared", the plane reaches ORB and the page opens your trips.
- With reduced motion switched on, the same states are shown with simple fades instead of movement.

The rules behind it (in `src/server/otp.ts`)

Rule	Value
Code	6 random digits from Node's cryptographic random generator
Stored as	An HMAC-SHA256 hash keyed with `AUTH_SECRET`; the code itself is never saved
Lifetime	10 minutes, and it works once
Wrong guesses	5, then the code is locked and a new one must be sent
Resend	After 30 seconds; at most 5 codes per attempt, and each new code replaces the old one
Per address	At most 6 new attempts an hour
Comparison	Constant-time, so response timing reveals nothing

The endpoints are `POST /api/auth/register` and `POST /api/auth/login` (step one, which return a code ticket), `POST /api/auth/otp/verify` (step two, which sets the session cookie) and `POST /api/auth/otp/resend`.

How a request flows through the app

```
Browser ──► Next.js page (server component) ──► repository ──► Prisma ──► PostgreSQL
   │                                              │
   │                                              └─ database down? ──► bundled catalog
(src/data/catalog.ts)
   │
   └─► client components (globe, animations, forms) ──► /api/* route handlers ──► validation (Zod)
                                                                                  ──► services /
repositories
                                                                                  ──► JSON response
```

- **Pages** are React Server Components: they fetch data on the server and send finished HTML. Anything interactive (the globe, GSAP animations, forms) is a client component marked `"use client"`.
- **API routes** in `src/app/api/**/route.ts` validate input with Zod, call a repository or service, and reply with JSON. Errors always look like `{ "error": { "code", "message", ... } }`.
- **Repositories** (`src/server/repositories`) are the only code that talks to the database.
- **Services** (`src/server/services`) talk to outside APIs: geocoding and attractions.
- **Sessions** are a signed cookie (`lgt_session`, HMAC-SHA256, 30 days, `httpOnly`). There is no session table; the signature proves the cookie wasn't forged.

Scripts

Script	What it does
`npm run dev`	Development server with hot reload on http://localhost:3000
`npm run build` / `npm start`	Production build, then serve it
`npm run typecheck` / `npm run lint`	TypeScript and ESLint checks

Script	What it does
`npm run db:local` / `npm run db:local:stop`	Start / stop the embedded Postgres (no Docker) on port 5433
`npm run db:up` / `npm run db:down`	Start / stop Postgres in Docker instead
`npm run db:setup`	Apply all migrations and seed the catalog
`npm run db:migrate`	Create a new migration after changing `prisma/schema.prisma`
`npm run db:seed` / `npm run db:studio`	Seed only / open Prisma Studio to browse the data
`npm run cesium:copy`	Re-copy the CesiumJS build into `public/cesium`
`npm run survey:build [slug]`	Rebuild the destination pages' terrain (`public/survey/*.json`) from AWS Terrain Tiles; needs internet

Project structure

```
prisma/                 schema.prisma (the data model), migrations/, seed.ts
scripts/                copy-cesium-assets.mjs, local-postgres.mjs, build-survey.ts
public/cesium/          the CesiumJS build (copied on install, not committed)
public/survey/          each destination's terrain heightfield (built by build-survey.ts)
docs/PROJECT_GUIDE.md   everything used in the project, in detail
src/
  app/                  pages and API routes (Next.js App Router)
    page.tsx            home (the 3D journey)
    destinations/       atlas and destination pages
    tours/              tours page and each tour's page
    book/               booking form and confirmation
    account/            sign in, sign up (with the email code), your trips
    api/                JSON endpoints (places, destinations, packages, bookings, auth, health)
    globals.css         the design system and every section's styles
  components/           shared UI: layout (nav, footer), account (auth pass, code step),
                        transitions (jet and boarding), tours, ui
  features/             the big interactive pieces
    globe/              CesiumJS engine: viewer, camera, markers, route, 3D city
    journey/            the home page's scroll story and its panels
    destinations/       the destinations atlas and reel, and the orbital survey (survey/)
    tours/              tours page sections and tour page sections (incl. the WebGL flyby)
    booking/            the booking terminal: check-in form, pass, the page-change curtain (curtain/,
drawn in
                        a Web Worker), confirmation
    account/            the login page's WebGL Earth horizon
    location/           browser location and time-zone fallback
  data/catalog.ts       the curated destinations, attractions and tours (source for the seed)
  lib/                  shared helpers: types, formatting, validation, geodesy, env, GSAP setup, themes
  server/               server-only code: database, auth, email codes, mail, repositories, services
  generated/prisma/     the generated Prisma client (created by `prisma generate`)
```

Degraded modes (by design)

Missing	What happens
Cesium ion token	The globe still renders with NASA imagery, no terrain or buildings; a notice says which variable to set.
Photorealistic tiles (limit or outage)	Terrain plus OSM Buildings draw the city instead; the notice says why.
Location permission	"Enter your location manually"; meanwhile the route starts from your time zone's main city, labelled approximate.
Database	Destinations and tours come from `src/data/catalog.ts`; booking and account APIs answer `503` with the exact command to run.
SMTP (development)	Codes are printed in the terminal and shown under the code boxes.
SMTP (production)	Sign-up and sign-in explain that email isn't set up; no code is ever shown on screen.
WebGL2	The jet and flyby animations are skipped or replaced by a simple airplane glyph; the destination survey is drawn with Canvas 2D; everything else works.
A destination's survey file	A generated landscape stands in for the real terrain.
Reduced motion	Animations become simple fades; the pinned scroll sequences show their final state.
Web Workers / OffscreenCanvas	The booking curtain's flaps are drawn on the page instead of in a worker.

Troubleshooting

- **"PostgreSQL is not reachable"**: start it with `npm run db:local` (keep that terminal open), then run `npm run db:setup` once.
- **"The database is reachable but not migrated"**: run `npm run db:setup`.
- **Credentials rejected on port 5433**: another Postgres is using the port; stop it or change `PGPORT`.
- **Port 3000 in use**: another dev server is running; stop it or open the one that's running.
- **No code email arrives**: check the terminal running `npm run dev` for the code box (SMTP blank) or for an `[otp] sending the code failed` error (SMTP set but wrong). Gmail needs an App Password, not your normal one.
- **The globe is plain blue with no city**: set `NEXT_PUBLIC_CESIUM_ION_TOKEN` and restart. `/api/health` reports what is configured.

Content and credits

Destinations, hotels, itineraries and prices are sample data. Coordinates come from Wikipedia and Wikidata. Photos are hotlinked from Wikimedia Commons. Earth imagery: NASA GIBS (Blue Marble, VIIRS city lights). The About page's imagery ships with the site in `public/about/`: the Moon from the NASA Scientific Visualization Studio CGI Moon Kit, the Earth from NASA Blue Marble Next Generation, Black Marble and Visible Earth (all public domain), the Landsat 8 3D model from NASA 3D Resources (public domain), and "Kullu Valley from Rohtang Pass, India" by

Vyacheslav Argenberg (CC BY 4.0, cropped and colour-graded), credited on the page. Photorealistic 3D Tiles © Google, via Cesium ion; terrain, imagery and fallback buildings via Cesium ion; building data © OpenStreetMap contributors. Keep the credits Cesium shows in the corner of the globe visible. Cesium ion's free Community plan is for personal and non-commercial use.